



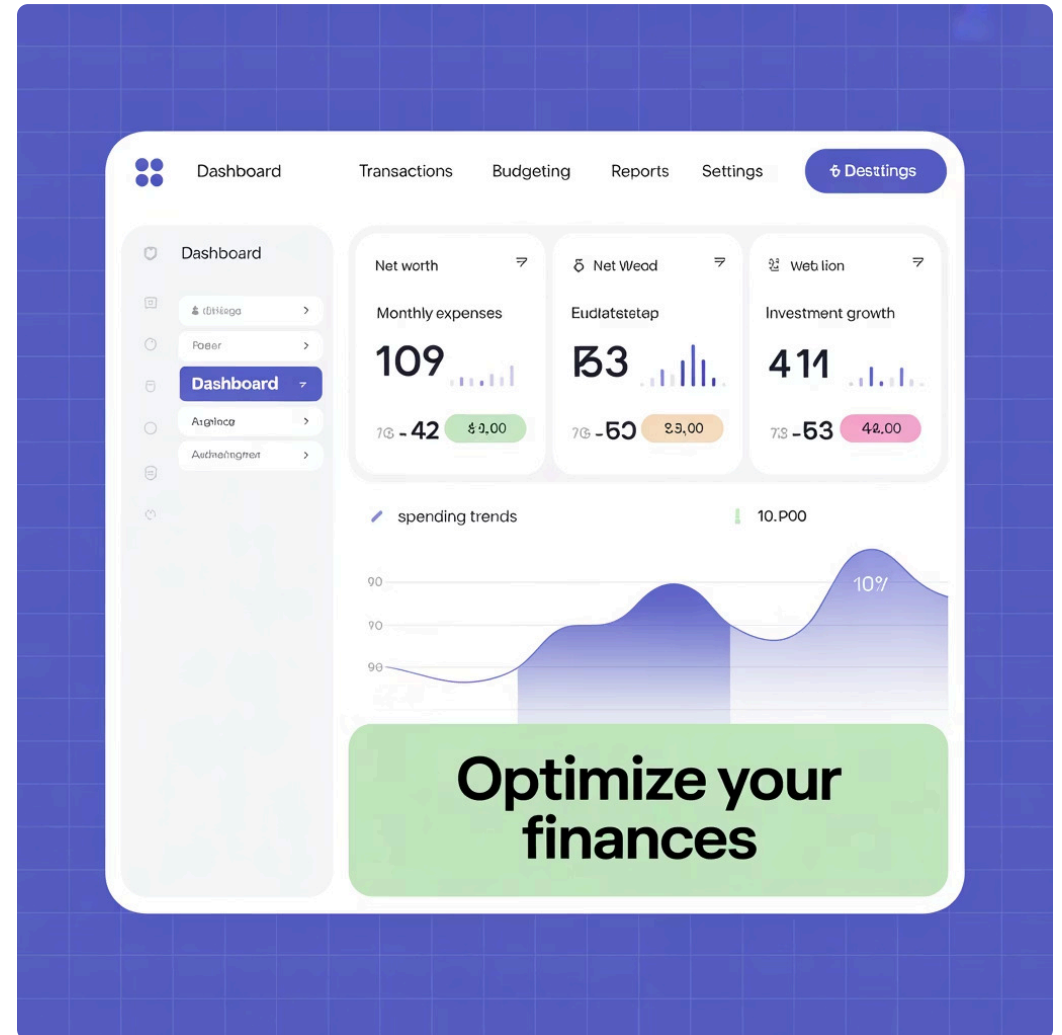
# Eventos en JavaScript

Los eventos son el corazón de la interactividad web. Descubre cómo JavaScript puede responder a las acciones del usuario y crear experiencias dinámicas que transforman páginas estáticas en aplicaciones interactivas.

# ¿Qué son los eventos?

Un evento es cualquier acción que ocurre en una página web. Desde el momento en que un usuario hace clic en un botón hasta cuando escribe en un campo de texto, cada interacción genera un evento que JavaScript puede detectar y procesar.

Los eventos son la base de la programación interactiva en JavaScript. Nos permiten crear páginas web que respondan de manera inmediata a las acciones del usuario, proporcionando feedback instantáneo y mejorando significativamente la experiencia de usuario.



# Tipos de eventos más comunes



## click

Se activa cuando el usuario hace clic en un elemento. Es el evento más utilizado para botones y enlaces interactivos.



## keydown

Se dispara al presionar cualquier tecla del teclado. Útil para crear atajos de teclado y validaciones en tiempo real.



## input

Se activa cada vez que cambia el valor de un campo de entrada. Perfecto para búsquedas en tiempo real y validaciones.



## submit

Se ejecuta al enviar un formulario. Esencial para procesar datos antes de enviarlos al servidor.

# addEventListener: Tu herramienta principal

El método `addEventListener` es la forma moderna y recomendada de gestionar eventos en JavaScript. Permite asociar funciones específicas a eventos concretos, manteniendo el código organizado y escalable.

## Sintaxis básica:

```
elemento.addEventListener("evento", función);
```

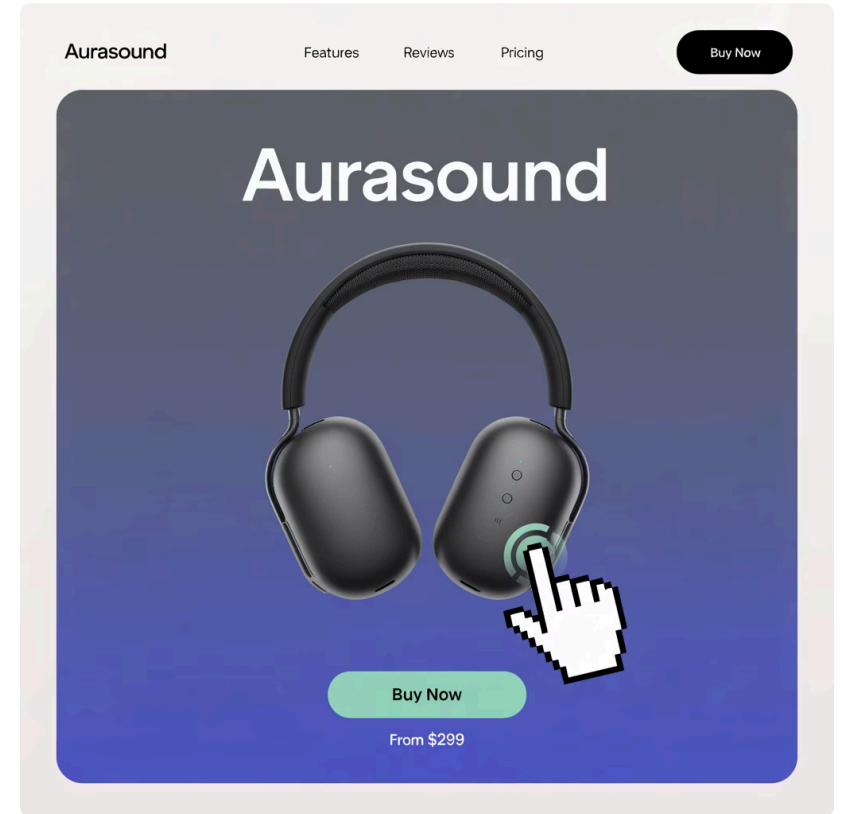
Esta sintaxis permite agregar múltiples escuchadores al mismo elemento y proporciona mayor flexibilidad que los métodos tradicionales como `onclick`.

# Ejemplo práctico: Click en botón

```
const boton = document.getElementById("miBoton");

boton.addEventListener("click", () => {
  console.log("Se hizo click en el botón");
  alert("¡Botón presionado!");
});
```

Este código demuestra cómo capturar el evento click de un botón. Cuando el usuario hace clic, se ejecuta la función que muestra un mensaje en la consola y una alerta.



El evento click es instantáneo y permite crear interfaces reactivas que responden inmediatamente a las acciones del usuario.

# Diferencias entre input y change

## Evento input

Se dispara **cada vez** que cambia el contenido del campo, incluso mientras el usuario sigue escribiendo. Es ideal para:

- Búsquedas en tiempo real
- Contadores de caracteres
- Validaciones instantáneas

## Evento change

Se activa **solo cuando** el elemento pierde el foco y su valor ha cambiado. Perfecto para:

- Validaciones finales
- Guardado automático
- Actualizaciones de bases de datos



# event.preventDefault(): Controlando el comportamiento

El método `preventDefault()` es fundamental para controlar el comportamiento por defecto de los elementos HTML. Su uso más común es en formularios, donde previene que la página se recargue automáticamente.

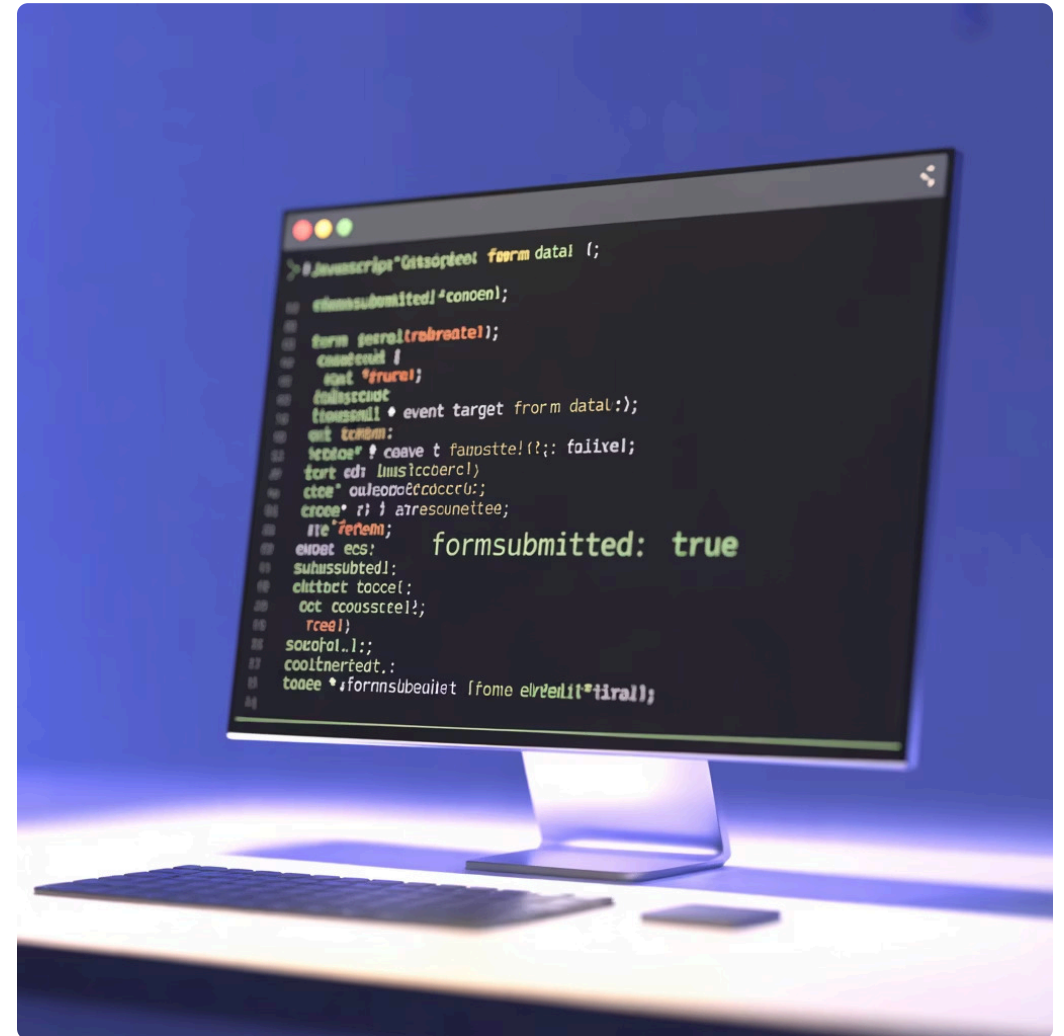
```
form.addEventListener("submit", (event) => {  
  event.preventDefault();  
  // Ahora podemos procesar el formulario sin recargar  
  console.log("Procesando datos...");  
});
```

 **¿Por qué es importante?** Sin `preventDefault()`, los formularios recargan la página por defecto, perdiendo el estado de la aplicación y interrumpiendo la experiencia del usuario.

# Accediendo a los valores: event.target.value

La propiedad `event.target.value` nos permite acceder al valor actual del elemento que disparó el evento. Es especialmente útil con campos de entrada donde necesitamos procesar la información que introduce el usuario.

```
inputEdad.addEventListener("input", (event) => {  
  const edad = event.target.value;  
  console.log("Edad:", edad);  
  
  if (edad >= 18) {  
    mensaje.textContent = "Eres mayor de edad";  
  }  
});
```



Esta técnica permite crear validaciones dinámicas y respuestas inmediatas a la entrada del usuario, mejorando la usabilidad de nuestras aplicaciones web.

# Stay in the Loop

Get exclusive updates and promotions  
delivered straight to your inbox

Enter your email address

Enter your address



Subscribe

## Ejercicio práctico: Formulario interactivo

Vamos a crear un formulario que muestre un saludo personalizado mientras el usuario escribe su nombre. Este ejercicio combina varios conceptos de eventos para crear una experiencia interactiva completa.

01

### HTML básico

Crear un formulario con un campo de nombre y un área para mostrar el saludo dinámico.

02

### Evento input

Escuchar cada cambio en el campo de texto para actualizar el saludo en tiempo real.

03

### Evento submit

Procesar el envío del formulario sin recargar la página usando `preventDefault()`.

# Resultado del ejercicio

## Código JavaScript completo:

```
const form = document.getElementById("formulario");
const inputNombre = document.getElementById("nombre");
const saludo = document.getElementById("saludo");

inputNombre.addEventListener("input", (event) => {
  saludo.textContent = `Hola, ${event.target.value}`;
});

form.addEventListener("submit", (event) => {
  event.preventDefault();
  alert(`Formulario enviado: ${inputNombre.value}`);
});
```

## Funcionalidades logradas:



El saludo se actualiza mientras el usuario escribe



El formulario no recarga la página al enviarse



Se muestra una confirmación con el nombre ingresado

¡Felicidades! Has creado tu primera aplicación interactiva con eventos en JavaScript. Este es el primer paso hacia interfaces web más dinámicas y atractivas.